

Technische Analyse - Pre-workout Webshop

1. Scope

In Scope

- Full stack webshop voor pre-workout supplementen.
- Figma-level UI designs.
- UML diagrams.
- Database ERD (Entity-Relationship Diagram).
- API contracten (specificaties voor de communicatie tussen frontend en backend).
- Productcatalogus module: functionaliteit voor het weergeven van een lijst met producten, inclusief filtering en sortering.
- Productdetail module: functionaliteit voor het weergeven van gedetailleerde informatie over een specifiek product.
- Winkelmand module: functionaliteit voor het toevoegen, verwijderen en aanpassen van producten in de winkelmand.
- Checkout module: functionaliteit voor het doorlopen van het bestelproces, inclusief adresgegevens en verzendopties.
- Betaling module: integratie met betaalproviders voor het verwerken van transacties.
- Account module: functionaliteit voor gebruikersregistratie, login, profielbeheer en ordergeschiedenis.
- Admin dashboard module: interface voor beheerders om producten, orders en gebruikers te beheren.
- API-laag: de backend service die de data en functionaliteit beschikbaar stelt aan de frontend.

Out of Scope

- Specifieke implementatiedetails van de betalingsprovider (bv. interne werking van Stripe, Mollie, etc.).
- Gedetailleerde marketingstrategieën (bv. SEO-optimalisatie, advertentiecampagnes).
- Fysieke distributie van producten (bv. magazijnbeheer, logistiek).

2. Assumptions

- De definitie van 'normale belasting' voor de laadtijd van het productoverzicht is een gemiddelde laadtijd van minder dan 2 seconden onder een gesimuleerde belasting van 100 gelijktijdige gebruikers.
- De ondersteunde betaalmethoden naast Bancontact en Creditcard zijn iDEAL en PayPal.
- Annuleren van een bestelling na het plaatsen ervan maar voor betaling resulteert in het automatisch verwijderen van de bestelling uit het systeem en het vrijgeven van eventueel gereserveerde voorraad.
- De beveiliging van de API-laag vereist naast authenticatie (bv. JWT) en autorisatie (rol-gebaseerde toegang) ook rate limiting en input validation op alle endpoints.
- De 'voorraadstatus' die getoond wordt op de productkaart wordt bepaald door de actuele voorraadhoeveelheid in de database. Een status 'Op voorraad' wordt getoond indien de voorraad > 0, 'Beperkt' indien 0 < voorraad <= 5, en 'Niet op voorraad' indien voorraad = 0.

3. Open Questions

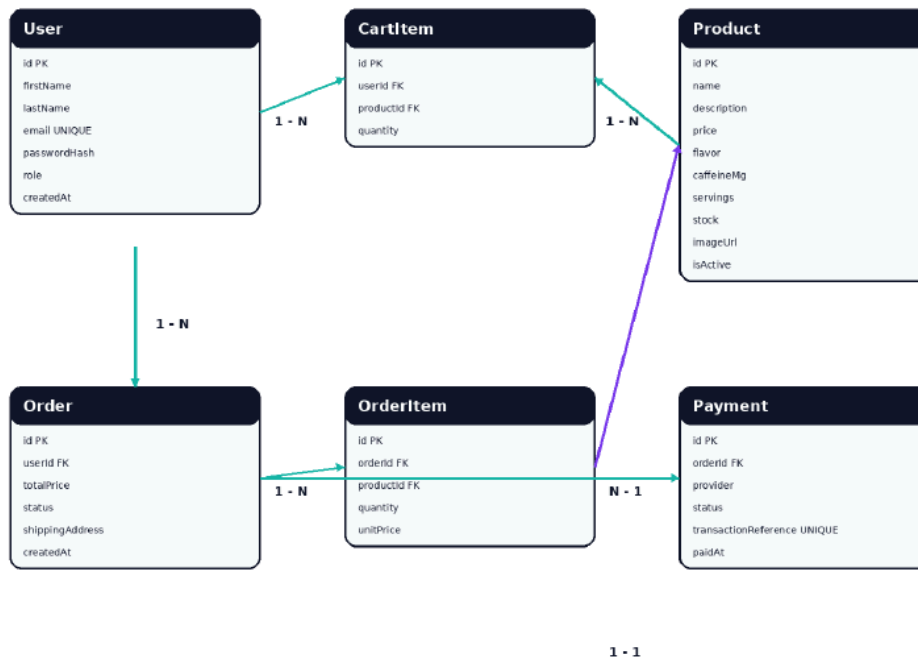
- Wat is de exacte definitie van 'normale belasting' voor de laadtijd van het productoverzicht? (bv. aantal gelijktijdige gebruikers, gemiddelde reactietijd in milliseconden).
- Welke specifieke betaalmethoden naast Bancontact en Creditcard worden ondersteund? (bv. iDEAL, PayPal, SEPA, etc.).
- Hoe wordt omgegaan met het annuleren van een bestelling na het plaatsen ervan maar voor betaling? (bv. automatische annulering, handmatige goedkeuring, notificatie naar klant/admin).
- Zijn er specifieke vereisten voor de beveiliging van de API-laag buiten de genoemde authenticatie en autorisatie? (bv. encryptie van data in transit, bescherming tegen specifieke OWASP Top 10 kwetsbaarheden).

- Wat zijn de criteria voor het bepalen van de 'voorraadstatus' die getoond wordt op de productkaart? (bv. drempelwaarden voor 'beperkt', omgang met backorders).

4. Domain Model

Database ERD

De database bestaat uit gebruikers, producten, winkelmanditems, bestellingen, orderregels en betalingen. De belangrijkste relaties zijn User 1-N Order, Order 1-N OrderItem, Product 1-N OrderItem, User 1-N CartItem en Order 1-1 Payment.



User

Veld	Type	Constraints	Testcases
id	UUID	notNull	missing
firstName	String	notNull, minLength:1, maxLength:255	empty, too_long, missing
lastName	String	notNull, minLength:1, maxLength:255	empty, too_long, missing
email	String	notNull, maxLength:255	empty, too_long, missing, invalid_value, duplicate_per_day
passwordHash	String	notNull, minLength:60, maxLength:60	empty, too_short, too_long, missing
role	Role	notNull	missing, invalid_value
createdAt	LocalDateTime	notNull	missing
updatedAt	LocalDateTime	notNull	missing

CartItem

Veld	Type	Constraints	Testcases
id	UUID	notNull	missing
userId	UUID	notNull	missing, invalid_value
productId	UUID	notNull	missing, invalid_value

Veld	Type	Constraints	Testcases
quantity	Integer	notNull, minLength:1	empty, too_short, missing, invalid_value
createdAt	LocalDateTime	notNull	missing
updatedAt	LocalDateTime	notNull	missing

Product

Veld	Type	Constraints	Testcases
id	UUID	notNull	missing
name	String	notNull, minLength:1, maxLength:255	empty, too_long, missing
description	String	notNull, minLength:1	empty, too_long, missing
price	BigDecimal	notNull, minLength:0	empty, too_short, missing, invalid_value
flavor	String	maxLength:255	too_long, invalid_value
caffeineMg	Integer	minLength:0	too_short, invalid_value
servings	Integer	minLength:1	too_short, invalid_value
stock	Integer	notNull, minLength:0	empty, too_short, missing, invalid_value
imageUrl	String	maxLength:255	too_long, invalid_value
isActive	Boolean	notNull	missing, invalid_value
createdAt	LocalDateTime	notNull	missing
updatedAt	LocalDateTime	notNull	missing

Order

Veld	Type	Constraints	Testcases
id	UUID	notNull	missing
userId	UUID	notNull	missing, invalid_value
totalPrice	BigDecimal	notNull, minLength:0	empty, too_short, missing, invalid_value
status	OrderStatus	notNull	missing, invalid_value
shippingAddress	String	notNull, minLength:1	empty, too_long, missing
createdAt	LocalDateTime	notNull	missing
updatedAt	LocalDateTime	notNull	missing

OrderItem

Veld	Type	Constraints	Testcases
id	UUID	notNull	missing
orderId	UUID	notNull	missing, invalid_value
productId	UUID	notNull	missing, invalid_value
quantity	Integer	notNull, minLength:1	empty, too_short, missing, invalid_value
unitPrice	BigDecimal	notNull, minLength:0	empty, too_short, missing, invalid_value
createdAt	LocalDateTime	notNull	missing
updatedAt	LocalDateTime	notNull	missing

Payment

Veld	Type	Constraints	Testcases
id	UUID	notNull	missing
orderId	UUID	notNull	missing, invalid_value
provider	PaymentProvider	notNull	missing, invalid_value
status	PaymentStatus	notNull	missing, invalid_value
transactionReference	String	notNull, maxLength:255	empty, too_long, missing, invalid_value, duplicate_per_day
paidAt	LocalDateTime		invalid_value
createdAt	LocalDateTime	notNull	missing
updatedAt	LocalDateTime	notNull	missing

Role

Veld	Type	Constraints	Testcases
value	enum_value	notNull	missing, invalid_value

OrderStatus

Veld	Type	Constraints	Testcases
value	enum_value	notNull	missing, invalid_value

PaymentProvider

Veld	Type	Constraints	Testcases
value	enum_value	notNull	missing, invalid_value

PaymentStatus

Veld	Type	Constraints	Testcases
value	enum_value	notNull	missing, invalid_value

Enums

- **Role:** ADMIN , CUSTOMER
- **OrderStatus:** PENDING , PROCESSING , SHIPPED , DELIVERED , CANCELLED
- **PaymentProvider:** STRIPE , PAYPAL
- **PaymentStatus:** PENDING , SUCCESS , FAILED , REFUNDED

5. API Design

5.1 Error Formaat

```
{
  "correlationId": "string",
  "code": "string",
  "message": "string",
  "fieldErrors": [
    {
      "field": "string",
      "message": "string"
    }
  ]
}
```

5.2 Endpoints

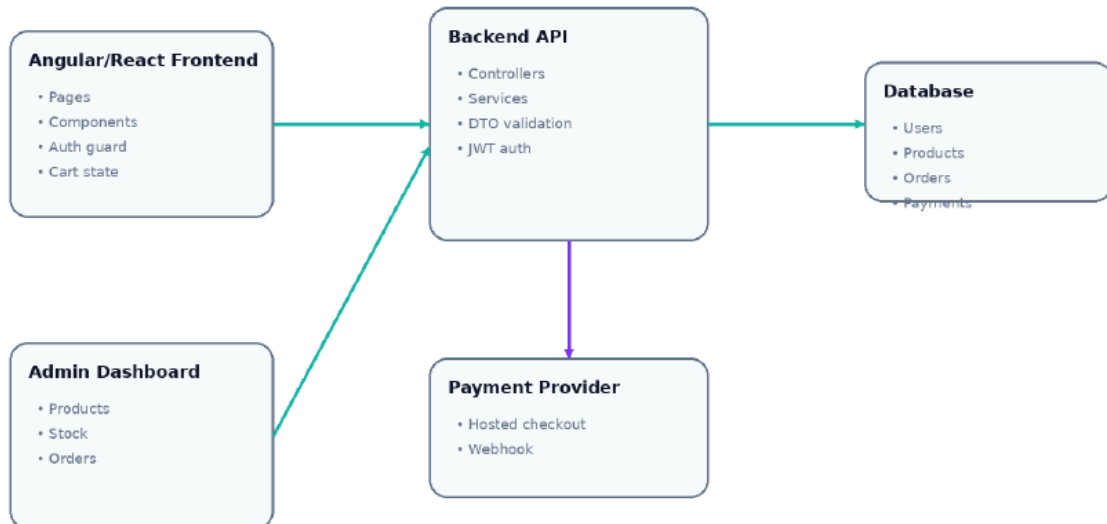
GET /api/products — Haal een lijst met producten op, met filter- en sorteeropties.

| Veld | Waarde

6. Backend Design

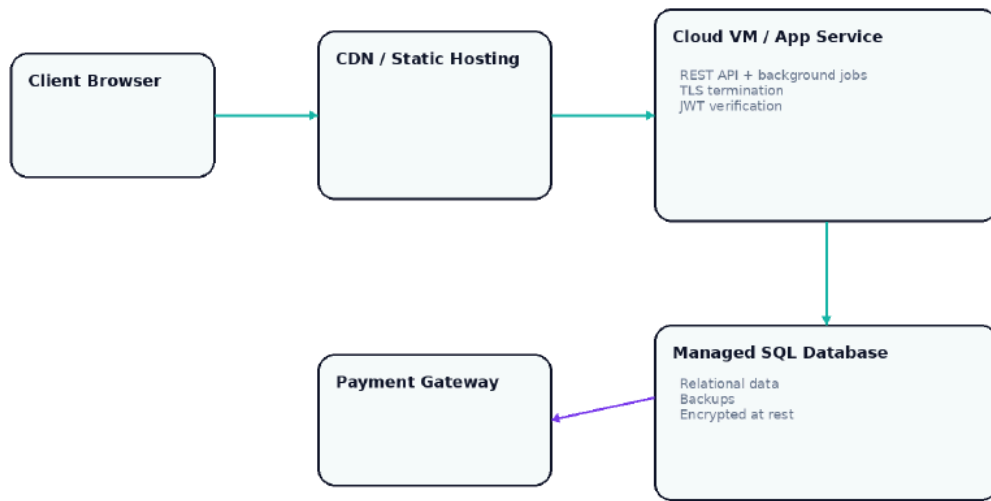
Component diagram

Dit diagram toont de logische applicatiecomponenten en hun verantwoordelijkheden.



Deployment diagram

Dit diagram toont een mogelijke cloud deployment met client, CDN, app service, database en payment gateway.



De backend van de Pre-workout Webshop volgt een gelaagde architectuur, bestaande uit de volgende lagen:

- **Controller Laag:** Verantwoordelijk voor het ontvangen van inkomende HTTP-verzoeken, het valideren van de input (vaak via DTO's) en het doorsturen van de verzoeken naar de Service Laag.
- **Service Laag:** Bevat de kern bedrijfslogica. Deze laag orkestreert de operaties, communiceert met de Repository Laag en handelt domein-specifieke validaties en uitzonderingen af.
- **Repository Laag:** Verantwoordelijk voor de interactie met de persistente gegevensopslag (bijvoorbeeld een database). Deze laag voert CRUD-operaties uit op de domeinobjecten.

Hieronder volgt een gedetailleerde beschrijving van de klassen per module:

Product Module

Klasse	Verantwoordelijkheid
<code>ProductController</code>	Behandelt inkomende HTTP-verzoeken voor productgerelateerde operaties.
<code>ProductService</code>	Bevat de bedrijfslogica voor het beheren van producten, inclusief zoeken, filteren en sorteren.
<code>ProductRepository</code>	Verantwoordelijk voor de interactie met de productgegevensopslag.
<code>Product</code>	Representeert een product in het domeinmodel.
<code>ProductDTO</code>	Data Transfer Object voor productinformatie in API-responses.
<code>ProductDetailDTO</code>	Data Transfer Object voor gedetailleerde productinformatie.
<code>ProductListResponseDTO</code>	Data Transfer Object voor de gepagineerde lijst van producten.
<code>CreateProductRequestDTO</code>	Data Transfer Object voor het aanmaken van een nieuw product.
<code>UpdateProductRequestDTO</code>	Data Transfer Object voor het bijwerken van een bestaand product.
<code>ProductNotFoundException</code>	Exception die wordt gegooid wanneer een product niet wordt gevonden.
<code>ProductSearchValidator</code>	Valideert de zoek- en filterparameters voor producten.

User Module

Klasse	Verantwoordelijkheid
<code>AuthController</code>	Behandelt inkomende HTTP-verzoeken voor gebruikersauthenticatie en -registratie.
<code>AuthService</code>	Bevat de bedrijfslogica voor gebruikersregistratie en -authenticatie.
<code>UserRepository</code>	Verantwoordelijk voor de interactie met de gebruikersgegevensopslag.
<code>User</code>	Representeert een gebruiker in het domeinmodel.
<code>UserResponseDTO</code>	Data Transfer Object voor gebruikersinformatie in API-responses.
<code>RegisterUserRequestDTO</code>	Data Transfer Object voor het registreren van een nieuwe gebruiker.
<code>LoginUserRequestDTO</code>	Data Transfer Object voor het inloggen van een gebruiker.
<code>LoginResponseDTO</code>	Data Transfer Object voor de login response, inclusief token.
<code>EmailAlreadyExistsException</code>	Exception die wordt gegooid wanneer een e-mailadres al in gebruik is.
<code>InvalidCredentialsException</code>	Exception die wordt gegooid bij ongeldige inloggegevens.
<code>UserRegistrationValidator</code>	Valideert de input voor gebruikersregistratie.
<code>UserLoginValidator</code>	Valideert de input voor gebruikerslogin.

Cart Module

Klasse	Verantwoordelijkheid
<code>CartController</code>	Behandelt inkomende HTTP-verzoeken voor winkelmandoperaties.
<code>CartService</code>	Bevat de bedrijfslogica voor het beheren van de winkelmand van een gebruiker.
<code>CartItemRepository</code>	Verantwoordelijk voor de interactie met de winkelmanditemgegevensopslag.
<code>CartItem</code>	Representeert een item in de winkelmand.
<code>CartResponseDTO</code>	Data Transfer Object voor de winkelmandinhoud.
<code>CartItemResponseDTO</code>	Data Transfer Object voor een individueel winkelmanditem.
<code>AddCartItemRequestDTO</code>	Data Transfer Object voor het toevoegen van een item aan de winkelmand.
<code>UpdateCartItemRequestDTO</code>	Data Transfer Object voor het bijwerken van de hoeveelheid van een winkelmanditem.
<code>InsufficientStockException</code>	Exception die wordt gegooid wanneer er onvoldoende voorraad is.
<code>CartItemNotFoundException</code>	Exception die wordt gegooid wanneer een winkelmanditem niet wordt gevonden.
<code>CartItemValidator</code>	Valideert de input voor winkelmanditems.

Order Module

Klasse	Verantwoordelijkheid
<code>OrderController</code>	Behandelt inkomende HTTP-verzoeken voor besteloperaties.
<code>OrderService</code>	Bevat de bedrijfslogica voor het plaatsen en beheren van bestellingen.
<code>OrderRepository</code>	Verantwoordelijk voor de interactie met de bestelgegevensopslag.
<code>OrderItemRepository</code>	Verantwoordelijk voor de interactie met de bestelitemgegevensopslag.
<code>Order</code>	Representeert een bestelling in het domeinmodel.
<code>OrderItem</code>	Representeert een item binnen een bestelling.
<code>OrderResponseDTO</code>	Data Transfer Object voor een bestelling.

Klasse	Verantwoordelijkheid
<code>OrderListResponseDTO</code>	Data Transfer Object voor een lijst van bestellingen.
<code>CreateOrderRequestDTO</code>	Data Transfer Object voor het aanmaken van een nieuwe bestelling.
<code>UpdateOrderStatusRequestDTO</code>	Data Transfer Object voor het bijwerken van de status van een bestelling.
<code>EmptyCartException</code>	Exception die wordt gegooid wanneer een bestelling wordt geplaatst met een lege winkelmand.
<code>OrderNotFoundException</code>	Exception die wordt gegooid wanneer een bestelling niet wordt gevonden.
<code>OrderCreationValidator</code>	Valideert de input voor het aanmaken van een bestelling.
<code>OrderStatusUpdateValidator</code>	Valideert de input voor het bijwerken van de bestelstatus.

Payment Module

Klasse	Verantwoordelijkheid
<code>PaymentController</code>	Behandelt inkomende HTTP-verzoeken voor betalingsgerelateerde operaties, inclusief webhooks.
<code>PaymentService</code>	Bevat de bedrijfslogica voor het verwerken van betalingen en het afhandelen van webhook callbacks.
<code>PaymentRepository</code>	Verantwoordelijk voor de interactie met de betalingsgegevensopslag.
<code>Payment</code>	Representeert een betaling in het domeinmodel.
<code>PaymentWebhookRequestDTO</code>	Data Transfer Object voor de payload van een betalingswebhook.
<code>PaymentWebhookValidator</code>	Valideert de input van een betalingswebhook.

Admin Module

Klasse	Verantwoordelijkheid
<code>AdminProductController</code>	Behandelt inkomende HTTP-verzoeken voor productbeheer door admins.
<code>AdminOrderController</code>	Behandelt inkomende HTTP-verzoeken voor bestelbeheer door admins.
<code>AdminProductService</code>	Bevat de bedrijfslogica voor productbeheer door admins.
<code>AdminOrderService</code>	Bevat de bedrijfslogica voor bestelbeheer door admins.
<code>AdminProductValidator</code>	Valideert de input voor productcreatie en -updates door admins.
<code>AdminOrderStatusUpdateValidator</code>	Valideert de input voor het bijwerken van bestelstatussen door admins.

Common Module

Klasse	Verantwoordelijkheid
<code>ApiError</code>	Standaardformaat voor foutmeldingen in API-responses.
<code>ApiErrorDTO</code>	Data Transfer Object voor de <code>ApiError</code> structuur.
<code>Role</code>	Enum die de rollen van gebruikers vertegenwoordigt (bv. USER, ADMIN).
<code>OrderStatus</code>	Enum die de statussen van bestellingen vertegenwoordigt.
<code>PaymentProvider</code>	Enum die de beschikbare betaalproviders vertegenwoordigt.
<code>PaymentStatus</code>	Enum die de statussen van betalingen vertegenwoordigt.
<code>GlobalExceptionHandler</code>	Centrale handler voor het afhandelen van exceptions en het genereren van <code>ApiError</code> responses.
<code>SecurityConfig</code>	Configureert beveiligingsinstellingen, inclusief authenticatie en autorisatie.
<code>JwtTokenProvider</code>	Verantwoordelijk voor het genereren en valideren van JWT-tokens.

Klasse	Verantwoordelijkheid
PasswordHasher	Verantwoordelijk voor het hashen en verifiëren van wachtwoorden.

7. Frontend Design

1. Samenvatting

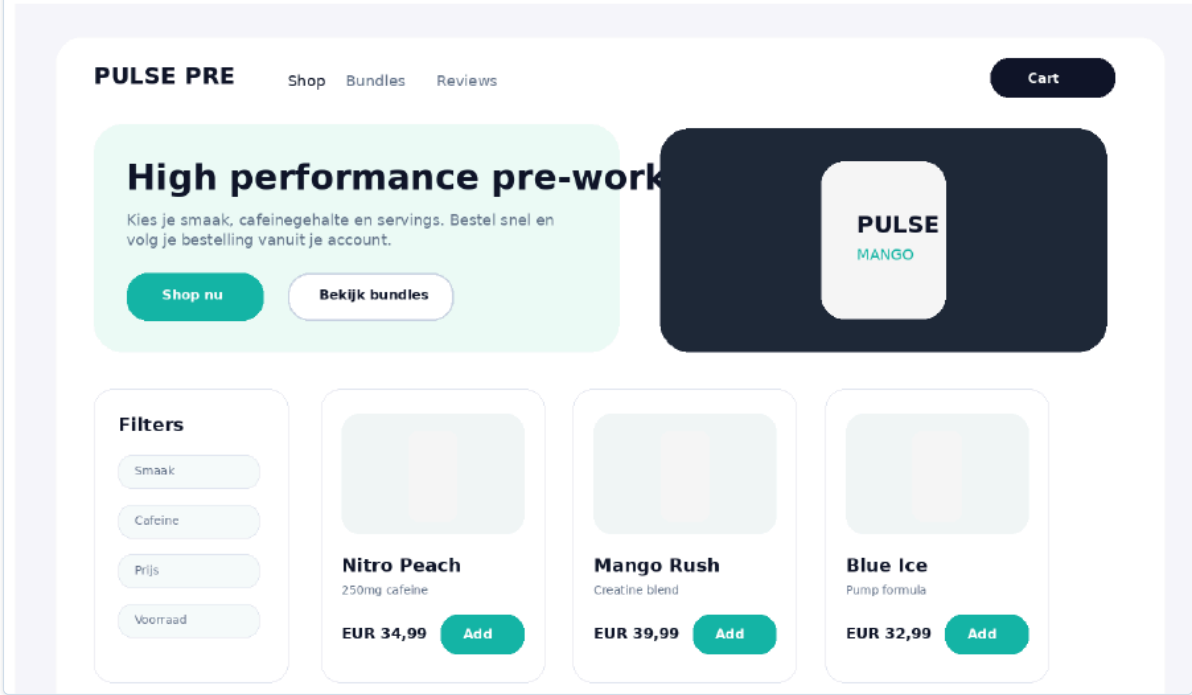
De applicatie is een traditionele webshop waarin klanten pre-workout producten kunnen zoeken, filteren, bekijken, toevoegen aan hun winkelmand, afrekenen en hun bestelgeschiedenis opvolgen. Admins beheren producten, voorraad en bestellingen.

Belangrijkste modules: productcatalogus, productdetail, winkelmand, checkout, betaling, account, admin dashboard, orderbeheer en API-laag.

2. Figma-level UI designs

De onderstaande schermen tonen een realistisch visueel ontwerp voor de belangrijkste klantflows. De stijl gebruikt duidelijke productcards, sterke CTA-knoppen, filterblokken en een checkout met gescheiden invoer en order summary.

Homepage + shop overzicht



Productdetailpagina

PULSE PRE

MANGO

Mango Rush Pre-Workout

Explosieve focus, pump en energie voor zware trainingen. Transparante ingrediënten en duidelijke dosage.

Cafeïne250 mg

Servings30

SmaakMango

Voorraad18 stuks

EUR 39,99

Toevoegen

Favoriet

Businessregel: knop is disabled wanneer stock = 0 of gekozen hoeveelheid groter is dan voorraad.

Account

Cart

Checkout en order summary

Checkout

Verzendgegevens

Voornaam

Achternaam

Straat + nr

Postcode

Gemeente

E-mail

Betaalmethode

Bancontact

Creditcard

Order summary

Mango Rush2xEUR 79,98

Blue Ice1xEUR 32,99

TotaalEUR 116,92

Betaal nu

file:///Users/owennolis/Desktop/AI-SDLC/docs/technical-analysis/tmps6e49_2_.html

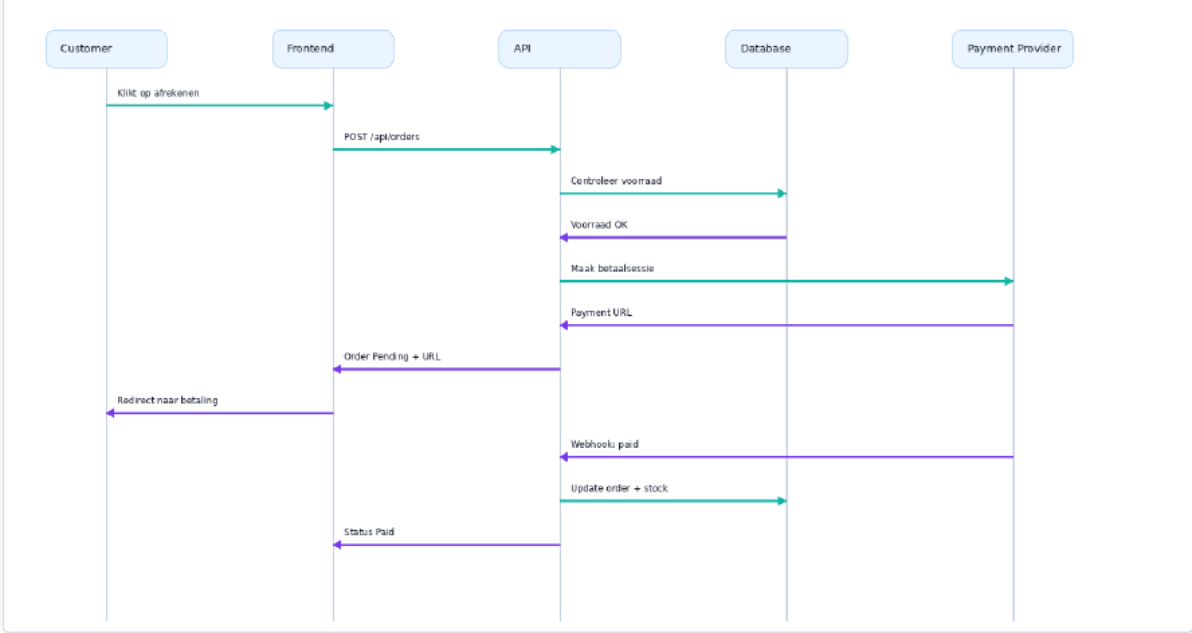
10/23

3. Uitgebreide UML diagrams

De UML diagrammen ondersteunen de analyse van gedrag, architectuurcomponenten en deployment.

Sequence diagram - checkout en betaling

Deze flow toont hoe klant, frontend, API, database en betalingsprovider samenwerken bij afrekenen.



/

Component	Verantwoordelijkheid
HomePage	Toont de homepage met hero sectie, product highlights en CTA's.
HeroSection	Toont de hero sectie met titel, ondertitel, CTA's en afbeelding.
ProductCardList	Toont een lijst met productkaarten.
ProductCard	Toont een individuele productkaart met basisinformatie en CTA.

/products

Component	Verantwoordelijkheid
ProductListPage	Toont de productcatalogus met filter- en sorteropties.
ProductFilterSidebar	Bevat filteropties voor producten (search, flavor, price, caffeine, inStock).
ProductSortDropdown	Dropdown voor sorteropties (priceAsc, priceDesc, nameAsc).
ProductCardList	Toont een lijst met productkaarten.
ProductCard	Toont een individuele productkaart met basisinformatie en CTA.
Pagination	Navigatie voor productpagina's.

/products/:id

Component	Verantwoordelijkheid
ProductDetailPage	Toont gedetailleerde informatie van een specifiek product.
ProductImageGallery	Toont productafbeeldingen.
ProductDetails	Toont producttitel, beschrijving, prijs en opties (caffeine, servings, flavor, stock).
AddToCartButton	Knop om product aan winkelmand toe te voegen.
FavoriteButton	Knop om product aan favorieten toe te voegen (indien van toepassing).

/cart

Component	Verantwoordelijkheid
CartPage	Toont de inhoud van de winkelmand.
CartItemList	Lijst met items in de winkelmand.
CartItem	Individueel item in de winkelmand met optie om hoeveelheid aan te passen of te verwijderen.
OrderSummary	Toont een overzicht van de bestelling (items, prijzen, totaal).
CheckoutButton	Knop om naar de checkout te gaan.

/checkout

Component	Verantwoordelijkheid
CheckoutPage	Behandelt het afrekenproces.
ShippingForm	Invoervelden voor verzendgegevens (naam, adres, postcode, gemeente, e-mail).
PaymentMethodSelector	Selectie van betaalmethoden (Bancontact, Creditcard).
OrderSummary	Toont een overzicht van de bestelling (items, prijzen, totaal).
PlaceOrderButton	Knop om de bestelling te plaatsen.

/order-history

Component	Verantwoordelijkheid
OrderHistoryPage	Toont de bestelgeschiedenis van de gebruiker.
OrderList	Lijst met eerdere bestellingen.
OrderItem	Individuele bestelling met status en details.

/login

Component	Verantwoordelijkheid
LoginPage	Pagina voor het inloggen van gebruikers.
LoginForm	Formulier voor e-mail en wachtwoord invoer.
RegisterLink	Link naar de registratiepagina.

/register

Component	Verantwoordelijkheid
RegisterPage	Pagina voor het registreren van nieuwe gebruikers.

Component	Verantwoordelijkheid
RegisterForm	Formulier voor gebruikersregistratie (voornaam, achternaam, e-mail, wachtwoord).

/admin

Component	Verantwoordelijkheid
AdminDashboardPage	Hoofdpagina van het admin dashboard.
AdminNavigation	Navigatie voor admin secties (producten, orders).

/admin/products

Component	Verantwoordelijkheid
AdminProductListPage	Toont een lijst met alle producten voor beheer.
AdminProductTable	Tabel met producten, inclusief bewerings- en verwijderingsopties.
AddProductButton	Knop om een nieuw product toe te voegen.

/admin/products/new

Component	Verantwoordelijkheid
AdminCreateProductPage	Pagina voor het aanmaken van een nieuw product.
ProductForm	Formulier voor het invoeren van productgegevens.

/admin/products/:id

Component	Verantwoordelijkheid
AdminEditProductPage	Pagina voor het bewerken van een bestaand product.
ProductForm	Formulier voor het bewerken van productgegevens.

/admin/orders

Component	Verantwoordelijkheid
AdminOrderListPage	Toont een lijst met alle bestellingen voor beheer.
AdminOrderTable	Tabel met bestellingen, inclusief statusupdates.

/admin/orders/:id

Component	Verantwoordelijkheid
AdminOrderDetailPage	Toont de details van een specifieke bestelling.
OrderStatusUpdater	Component om de status van een bestelling bij te werken.

*

Component	Verantwoordelijkheid
NotFoundPage	Toont een 404-pagina voor ongeldige routes.

8. Security & Privacy

8.1 Authenticatie

- **Gebruikersauthenticatie:** Klanten en admins authenticeren zich via `/api/auth/login`. Wachtwoorden worden gehasht opgeslagen met een robuust algoritme (bv. bcrypt) en gesalt.
- **Token-gebaseerde authenticatie:** Na succesvolle login wordt een JSON Web Token (JWT) uitgegeven. Dit token wordt meegestuurd in de `Authorization` header (Bearer token) voor alle beveiligde endpoints.
- **Sessiebeheer:** JWT's hebben een beperkte levensduur en kunnen worden verversd. Er wordt een mechanisme geïmplementeerd voor het intrekken van tokens bij uitloggen of beveiligingsincidenten.
- **Admin authenticatie:** Admins hebben een aparte rol en worden geauthenticeerd via dezelfde login endpoint, maar met specifieke permissies.

8.2 Autorisatie

- **Rolgebaseerde toegangscontrole (RBAC):**
 - **Klanten:** Kunnen producten bekijken, zoeken, filteren, toevoegen aan winkelmand, afrekenen en hun bestelgeschiedenis opvolgen (`/api/products`, `/api/cart`, `/api/orders/me`).
 - **Admins:** Kunnen producten, voorraad en bestellingen beheren (`/api/admin/products`, `/api/admin/products/{id}`, `/api/admin/orders/{id}/status`).
- **Endpoint-specifieke autorisatie:** Elk endpoint controleert of de geauthenticeerde gebruiker de benodigde rol en permissies heeft om de actie uit te voeren. Bijvoorbeeld, alleen admins kunnen `/api/admin/products` aanroepen.
- **Data-isolatie:** Klanten kunnen alleen hun eigen bestelgeschiedenis (`/api/orders/me`) inzien.

8.3 Privacyoverwegingen

- **Gegevensminimalisatie:** Alleen noodzakelijke persoonsgegevens worden verzameld tijdens het checkout-proces (voornaam, achternaam, adres, e-mail).
- **Gegevensopslag:** Persoonsgegevens worden veilig opgeslagen en alleen toegankelijk voor geautoriseerd personeel.
- **Betalingsgegevens:** Gevoelige betalingsgegevens (creditcardnummers) worden niet direct opgeslagen door de applicatie, maar afgehandeld door een externe, PCI-DSS-conforme betaalprovider. De webhook (`/api/payments/webhook`) ontvangt alleen transactie-ID's en statusupdates.
- **Wachtwoordbeveiliging:** Wachtwoorden worden gehasht en gesalt opgeslagen, nooit in platte tekst.
- **Toestemming:** Indien nodig, wordt expliciete toestemming gevraagd voor het gebruik van gegevens voor marketingdoeleinden.

9. Observability

9.1 Logging

- **Loglevels:** Gebruik van gestandaardiseerde loglevels (DEBUG, INFO, WARN, ERROR, FATAL).
- **Gestructureerde logging:** Logs worden in JSON-formaat opgeslagen voor eenvoudige parsing en analyse.
- **Contextuele informatie:** Elke log entry bevat minimaal:
 - `timestamp`: ISO 8601 formaat.
 - `level`: Loglevel.
 - `message`: Beschrijvende boodschap.
 - `correlation_id`: Zie sectie 9.3.
 - `user_id` (indien van toepassing): ID van de ingelogde gebruiker.
 - `endpoint` (indien van toepassing): De aangeroepen API endpoint.
 - `http_method` (indien van toepassing): GET, POST, PUT, DELETE.
 - `status_code` (indien van toepassing): HTTP status code van de response.

- `error_details` (indien van toepassing): Stack trace of specifieke foutmeldingen.
- **Concrete logvoorbeelden:**
 - **INFO:** `{"timestamp": "2023-10-27T10:00:00Z", "level": "INFO", "message": "User logged in successfully", "correlation_id": "abc123xyz789", "user_id": "user-456", "endpoint": "/api/auth/login", "http_method": "POST", "status_code": 200}`
 - **ERROR:** `{"timestamp": "2023-10-27T10:05:15Z", "level": "ERROR", "message": "Failed to retrieve product details", "correlation_id": "abc123xyz789", "user_id": "user-456", "endpoint": "/api/products/999", "http_method": "GET", "status_code": 404, "error_details": "Product with ID 999 not found"}`
 - **DEBUG:** `{"timestamp": "2023-10-27T10:10:30Z", "level": "DEBUG", "message": "Processing cart update", "correlation_id": "abc123xyz789", "user_id": "user-456", "endpoint": "/api/cart/items/123", "http_method": "PUT", "status_code": 200, "payload": {"quantity": 2}}`
 - **WARN:** `{"timestamp": "2023-10-27T10:15:45Z", "level": "WARN", "message": "Low stock warning for product", "correlation_id": "abc123xyz789", "product_id": "prod-789", "current_stock": 5}`

9.2 Metrics

- **Request Latency:** Meten van de responstijd per endpoint (bv. gemiddelde, P95, P99).
- **Error Rates:** Aantal fouten per endpoint, per type fout (bv. 4xx, 5xx).
- **Throughput:** Aantal requests per seconde per endpoint.
- **Resource Utilization:** CPU, geheugen, netwerkgebruik van de applicatie servers.
- **Database Metrics:** Query performance, connectiepool gebruik.
- **Business Metrics:**
 - Aantal succesvolle bestellingen.
 - Aantal items in winkelmandjes.
 - Aantal geregistreerde gebruikers.
- **Monitoring Tools:** Integratie met tools zoals Prometheus, Grafana, Datadog.

9.3 Correlation ID

- **Doel:** Het traceren van een enkele request door alle microservices en componenten heen.
- **Implementatie:**
 - Bij de initiële request naar de API Gateway of de eerste service wordt een unieke `correlation_id` gegenereerd (bv. UUID).
 - Deze `correlation_id` wordt meegestuurd in alle volgende interne requests (bv. via HTTP headers).
 - Alle logs die gerelateerd zijn aan deze request bevatten dezelfde `correlation_id`.
- **Voorbeeld:** Een klant voegt een product toe aan de winkelmand. De request gaat naar `/api/cart/items`. De `correlation_id` wordt gegenereerd. Als deze request vervolgens een interne call maakt naar een product service om de voorraad te checken, wordt dezelfde `correlation_id` meegestuurd. Alle logs van deze keten van events kunnen dan worden gefilterd op deze `correlation_id` om de volledige flow te analyseren.

10. Performance & Scalability

10.1 Performance-eisen

- **Productoverzicht laden:** Moet binnen 2 seconden laden bij normale belasting. Dit geldt voor de `/api/products` endpoint.
- **Productdetailpagina laden:** Moet binnen 1 seconde laden. Dit geldt voor de `/api/products/{id}` endpoint.
- **Winkelmandje bijwerken:** Moet binnen 500ms reageren. Dit geldt voor `/api/cart/items` (POST, PUT, DELETE).
- **Checkout proces:** De initiële stappen van het checkout proces (bv. verzendgegevens invoeren) moeten responsief zijn (< 1 seconde). De uiteindelijke orderbevestiging na betaling kan iets langer duren, maar de gebruiker moet feedback

krijgen dat de order wordt verwerkt.

10.2 Database-indexen

- **products tabel:**
 - `id` (Primary Key)
 - `name` (Voor full-text search en filtering)
 - `category` (Voor filtering)
 - `price` (Voor sortering en filtering)
 - `stock_quantity` (Voor filtering op beschikbaarheid)
- **users tabel:**
 - `id` (Primary Key)
 - `email` (Voor login en unieke identificatie)
- **orders tabel:**
 - `id` (Primary Key)
 - `user_id` (Voor het ophalen van bestellingen per gebruiker)
 - `order_date` (Voor sortering en filtering)
 - `status` (Voor filtering van bestellingen)
- **order_items tabel:**
 - `id` (Primary Key)
 - `order_id` (Foreign Key naar `orders`)
 - `product_id` (Foreign Key naar `products`)
- **cart_items tabel:**
 - `id` (Primary Key)
 - `user_id` (Voor het koppelen van winkelmand items aan gebruikers)
 - `product_id` (Foreign Key naar `products`)

10.3 Schaalbaarheid

- **Stateless Services:** De backend services (API endpoints) moeten stateless zijn, zodat ze eenvoudig horizontaal geschaald kunnen worden. Sessie-informatie wordt opgeslagen in een externe store (bv. Redis) of via JWT.
- **Database Schaalbaarheid:**
 - **Replicatie:** Gebruik van read replicas voor de database om leesintensieve operaties (bv. productoverzicht) te ontlasten.
 - **Sharding:** Indien de dataset extreem groot wordt, kan sharding overwogen worden.
- **Caching:**
 - **Productdata:** Veelvuldig opgevraagde productinformatie kan gecached worden (bv. in Redis) om database-loads te verminderen.
 - **API Gateway Caching:** Caching van statische of semi-statische API responses.
- **Asynchrone Verwerking:**
 - **Bestellingsverwerking:** Complexe of tijdrovende processen na een bestelling (bv. e-mail notificaties, voorraadupdates) kunnen asynchroon worden afgehandeld met behulp van message queues (bv. RabbitMQ, Kafka).
 - **Betalingswebhooks:** De `/api/payments/webhook` kan een snelle initiële respons geven en de verdere verwerking (bv. orderstatus update) asynchroon uitvoeren.
- **Load Balancing:** Gebruik van load balancers om verkeer te verdelen over meerdere instances van de applicatie services.
- **CDN (Content Delivery Network):** Voor het serveren van statische assets zoals productafbeeldingen, om de laadtijd te versnellen en de serverbelasting te verminderen.

- **Microservices Architectuur (optioneel):** Indien de complexiteit toeneemt, kan een microservices architectuur overwogen worden, waarbij verschillende functionaliteiten (bv. productcatalogus, winkelmand, orders) als aparte services worden ontwikkeld en geschaald.

11. Test Strategy

De teststrategie voor de Pre-workout Webshop is ontworpen om de kwaliteit, stabiliteit en betrouwbaarheid van de applicatie te waarborgen gedurende de gehele levenscyclus van de ontwikkeling. We hanteren een gelaagde aanpak, waarbij elke testlaag specifieke aspecten van de software valideert.

Unit Tests

Unit tests richten zich op het isoleren en testen van de kleinste testbare eenheden van de applicatie, doorgaans functies of methoden. Deze tests worden uitgevoerd door ontwikkelaars tijdens de implementatiefase om de correcte werking van individuele componenten te verifiëren.

- HomePage render
- ProductCard render
- ProductFilterSidebar render
- ProductSortDropdown render
- Pagination render
- ProductDetailPage render
- ProductImageGallery render
- ProductDetails render
- AddToCartButton render
- CartPage render
- CartItemList render
- CartItem render
- OrderSummary render
- CheckoutPage render
- ShippingForm render
- PaymentMethodSelector render
- OrderHistoryPage render
- OrderList render
- OrderItem render
- LoginPage render
- LoginForm render
- RegisterPage render
- RegisterForm render
- AdminDashboardPage render
- AdminNavigation render
- AdminProductListPage render
- AdminProductTable render
- AdminCreateProductPage render
- ProductForm render
- AdminEditProductPage render
- AdminOrderListPage render
- AdminOrderTable render
- AdminOrderDetailPage render
- OrderStatusUpdater render
- NotFoundPage render

- ApiErrorDisplay render
- LoadingSpinner render

Integration Tests

Integratietests valideren de interactie tussen verschillende modules of services. In de context van de Pre-workout Webshop richten deze tests zich voornamelijk op de API-endpoints en de communicatie tussen de frontend en de backend.

- GET /api/products → 200 OK: Verifieert dat de lijst met producten correct wordt opgehaald.
- GET /api/products/{id} → 200 OK: Verifieert dat een specifiek product correct wordt opgehaald op basis van zijn ID.
- POST /api/auth/register → 201 Created: Verifieert dat een nieuwe gebruiker succesvol kan worden geregistreerd.
- POST /api/auth/login → 200 OK: Verifieert dat een gebruiker succesvol kan inloggen.
- GET /api/cart → 200 OK: Verifieert dat de inhoud van de winkelmand correct wordt opgehaald.
- POST /api/cart/items → 201 Created: Verifieert dat een product succesvol aan de winkelmand kan worden toegevoegd.
- PATCH /api/cart/items/{id} → 200 OK: Verifieert dat de hoeveelheid van een item in de winkelmand succesvol kan worden bijgewerkt.
- DELETE /api/cart/items/{id} → 204 No Content: Verifieert dat een item succesvol uit de winkelmand kan worden verwijderd.
- POST /api/orders → 201 Created: Verifieert dat een nieuwe bestelling succesvol kan worden geplaatst.
- GET /api/orders/me → 200 OK: Verifieert dat de bestelgeschiedenis van de ingelogde gebruiker correct wordt opgehaald.
- POST /api/admin/products → 201 Created: Verifieert dat een nieuw product succesvol kan worden aangemaakt door een admin.
- PUT /api/admin/products/{id} → 200 OK: Verifieert dat een bestaand product succesvol kan worden bewerkt door een admin.
- PATCH /api/admin/orders/{id}/status → 200 OK: Verifieert dat de status van een bestelling succesvol kan worden bijgewerkt door een admin.

End-to-End (E2E) Tests

E2E-tests simuleren realistische gebruikersscenario's door de gehele applicatiestroom te doorlopen, van de gebruikersinterface tot de backend en eventuele externe integraties. Deze tests worden uitgevoerd in een omgeving die zo dicht mogelijk bij de productieomgeving ligt.

- Gebruiker navigeert naar de homepage, bekijkt producten, voegt een product toe aan de winkelmand, gaat naar de checkout, vult verzendgegevens in, plaatst de bestelling en bekijkt de bestelgeschiedenis.
- Gebruiker registreert een nieuw account, logt in, voegt een product toe aan de winkelmand en gaat naar de checkout.
- Admin logt in, navigeert naar de productbeheerpagina, voegt een nieuw product toe, bewerkt een bestaand product en verwijdert het product.
- Admin logt in, navigeert naar de bestelbeheerpagina, bekijkt de details van een bestelling en werkt de status van de bestelling bij.

12. Acceptance Criteria

AC-ID	REQ	Gegeven	Wanneer	Dan	Testtype
AC-001-1	REQ-001	Een gebruiker is ingelogd en er zijn pre-workout producten beschikbaar in de database.	De gebruiker navigeert naar de productlijstpagina en voert 'Whey Protein' in het zoekveld in.	De API retourneert een HTTP 200 met een lijst van producten waarvan de naam 'Whey Protein' bevat, en de productcard stijl is toegepast.	integration

AC-ID	REQ	Gegeven	Wanneer	Dan	Testtype
AC-001-2	REQ-001	Een gebruiker is ingelogd en er zijn pre-workout producten beschikbaar met verschillende 'caffeineMg' waarden.	De gebruiker filtert de productlijst op 'caffeineMg' tussen 100 en 200.	De API retourneert een HTTP 200 met een lijst van producten waarbij 'caffeineMg' tussen 100 en 200 ligt.	integration
AC-001-3	REQ-001	Een gebruiker is ingelogd en er is een product met ID 'prod-123' beschikbaar.	De gebruiker voegt 2 stuks van product 'prod-123' toe aan de winkelmand.	De API retourneert een HTTP 201 met een CartItemResponse die de toegevoegde items met de correcte hoeveelheid en product-ID bevat.	integration
AC-001-4	REQ-001	Een gebruiker is ingelogd en heeft producten in de winkelmand.	De gebruiker navigeert naar de checkoutpagina en voltooit de bestelling met geldige verzendgegevens en betaalmethode.	De API retourneert een HTTP 201 met een OrderResponse die de gecreëerde bestelling bevat, en de winkelmand is leeggemaakt.	integration
AC-001-5	REQ-001	Een gebruiker is ingelogd en heeft eerder bestellingen geplaatst.	De gebruiker navigeert naar de bestelgeschiedenispagina.	De API retourneert een HTTP 200 met een OrderListResponse die een lijst van de geplaatste bestellingen van de gebruiker bevat.	integration
AC-001-6	REQ-001	Een gebruiker is niet ingelogd.	De gebruiker probeert een product toe te voegen aan de winkelmand.	De API retourneert een HTTP 401 met een ApiError die aangeeft dat authenticatie vereist is.	integration
AC-002-1	REQ-002	Een gebruiker met de rol 'ADMIN' is ingelogd.	De admin voegt een nieuw product toe via POST /api/admin/products met de volgende gegevens: name='Test Product', description='Een test product', price=19.99, stock=50.	De API retourneert een HTTP 201 met een ProductResponse die het nieuw aangemaakte product bevat, inclusief de verstrekte details.	integration
AC-002-2	REQ-002	Een gebruiker met de rol 'ADMIN' is ingelogd en er is een product met ID 'prod-456' in de database.	De admin update de voorraad van product 'prod-456' naar 75 via PATCH /api/admin/products/prod-456 met body {stock: 75}.	De API retourneert een HTTP 200 met een ProductResponse die het bijgewerkte product bevat, met een voorraad van 75.	integration
AC-002-3	REQ-002	Een gebruiker met de rol 'ADMIN' is ingelogd en er is een bestelling met ID 'order-789' met status 'PENDING'.	De admin wijzigt de status van bestelling 'order-789' naar 'SHIPPED' via PATCH /api/admin/orders/order-789/status met body {status: 'SHIPPED'}.	De API retourneert een HTTP 200 met een OrderResponse die de bijgewerkte bestelling bevat, met status 'SHIPPED'.	integration
AC-002-4	REQ-002	Een gebruiker met de rol 'USER' is ingelogd.	De gebruiker probeert een nieuw product toe te voegen via POST /api/admin/products.	De API retourneert een HTTP 403 met een ApiError die aangeeft dat de gebruiker niet geautoriseerd is.	integration
AC-002-5	REQ-002	Een gebruiker met de rol 'ADMIN' is ingelogd en er is een product met ID 'prod-999' dat niet bestaat.	De admin probeert de voorraad van product 'prod-999' te updaten via PATCH	De API retourneert een HTTP 404 met een ApiError die aangeeft dat het product niet gevonden is.	integration

AC-ID	REQ	Gegeven	Wanneer	Dan	Testtype
			/api/admin/products/prod-999 met body {stock: 10}.		
AC-003-1	REQ-003	Er worden producten weergegeven op de productlijstpagina.	De pagina wordt geladen.	Elk product wordt weergegeven in een productcard met een duidelijke afbeelding, titel, prijs en een 'Toevoegen aan winkelmand' knop met een prominente stijl (bv. contrasterende kleur, duidelijke tekst).	e2e
AC-003-2	REQ-003	Een productcard wordt weergegeven.	De gebruiker hooft over de 'Toevoegen aan winkelmand' knop.	De knop toont een visuele feedback (bv. kleurverandering, lichte animatie) om aan te geven dat deze interactief is.	e2e
AC-004-1	REQ-004	Een gebruiker is ingelogd en heeft producten in de winkelmand.	De gebruiker navigeert naar de checkoutpagina.	De checkoutpagina toont een sectie met een overzicht van de bestelling, inclusief de namen, aantallen en prijzen van de producten in de winkelmand.	e2e
AC-004-2	REQ-004	Een gebruiker is ingelogd en heeft producten in de winkelmand.	De gebruiker navigeert naar de checkoutpagina.	De checkoutpagina toont een 'Order Summary' sectie met de totale prijs van de bestelling, inclusief eventuele verzendkosten en belastingen.	e2e
AC-004-3	REQ-004	Een gebruiker is ingelogd en heeft producten in de winkelmand.	De gebruiker probeert de checkout te voltooien zonder verzendgegevens in te vullen.	De applicatie toont een foutmelding die aangeeft dat de verzendgegevens verplicht zijn, en de bestelling wordt niet geplaatst.	e2e
AC-005-1	REQ-005	De applicatie wordt geopend.	De homepage wordt geladen.	De homepage bevat een hero sectie met een duidelijke titel (bv. 'Ontdek Onze Krachtige Pre-Workouts'), een ondertitel (bv. 'Maximaliseer je training'), minimaal één call-to-action knop (bv. 'Shop Nu') en een relevante achtergrondafbeelding.	e2e
AC-005-2	REQ-005	De homepage wordt weergegeven.	De gebruiker klikt op de 'Shop Nu' call-to-action knop in de hero sectie.	De gebruiker wordt doorgestuurd naar de productlijstpagina.	e2e
AC-006-1	REQ-006	Er is een product met ID 'prod-abc' in de database met de volgende details: name='Intense Energy', description='Boost je focus', flavor='Fruit Punch', caffeineMg=150, servings=30, stock=20, price=29.99, imageUrl='http://example.com/image.jpg'.	De gebruiker navigeert naar de productdetailpagina voor product 'prod-abc'.	De pagina toont de afbeelding 'http://example.com/image.jpg', de titel 'Intense Energy', de beschrijving 'Boost je focus', de opties 'Fruit Punch', '150mg cafeïne', '30 servings', de prijs '€29.99', en een 'Toevoegen aan winkelmand' knop en een 'Favoriet' knop.	integration
AC-006-2	REQ-006	Er is een product met ID 'prod-xyz' met een voorraad van 0 stuks.	De gebruiker bekijkt de productdetailpagina voor product 'prod-xyz'.	De 'Toevoegen aan winkelmand' knop is uitgeschakeld of toont een bericht 'Niet op voorraad'.	e2e

AC-ID	REQ	Gegeven	Wanneer	Dan	Testtype
AC-006-3	REQ-006	Er is een product met ID 'prod-def' met een voorraad van 5 stuks.	De gebruiker probeert 6 stuks van product 'prod-def' toe te voegen aan de winkelmand vanaf de productdetailpagina.	De applicatie toont een foutmelding dat de gevraagde hoeveelheid de beschikbare voorraad overschrijdt, en de hoeveelheid wordt niet bijgewerkt in de winkelmand.	e2e
AC-007-1	REQ-007	Een gebruiker is ingelogd en navigeert naar de checkoutpagina.	De gebruiker bekijkt de verzendgegevens sectie.	De sectie bevat invoervelden voor: voornaam, achternaam, straat + nummer, postcode, gemeente en e-mailadres.	e2e
AC-007-2	REQ-007	Een gebruiker is ingelogd en navigeert naar de checkoutpagina.	De gebruiker bekijkt de betaalmethoden sectie.	De sectie toont opties voor 'Bancontact' en 'Creditcard'.	e2e
AC-007-3	REQ-007	Een gebruiker is ingelogd en navigeert naar de checkoutpagina.	De gebruiker probeert de checkout te voltooien zonder het verplichte veld 'e-mailadres' in te vullen.	De applicatie toont een foutmelding die aangeeft dat het e-mailadres verplicht is, en de bestelling wordt niet geplaatst.	e2e
AC-007-4	REQ-007	Een gebruiker is ingelogd en navigeert naar de checkoutpagina.	De gebruiker probeert de checkout te voltooien met een ongeldig e-mailformaat (bv. 'test@.com').	De applicatie toont een foutmelding die aangeeft dat het e-mailadres ongeldig is, en de bestelling wordt niet geplaatst.	e2e
AC-008-1	REQ-008	Een gebruiker is ingelogd en heeft producten in de winkelmand.	De gebruiker navigeert naar de checkoutpagina.	De 'Order Summary' sectie toont een lijst van bestelde items, waarbij elk item de naam, het aantal en de prijs per item vermeldt.	e2e
AC-008-2	REQ-008	Een gebruiker is ingelogd en heeft producten in de winkelmand.	De gebruiker navigeert naar de checkoutpagina.	De 'Order Summary' sectie toont de totale prijs van de bestelling, berekend als de som van de prijzen van alle bestelde items.	e2e
AC-008-3	REQ-008	Een gebruiker is ingelogd en heeft 2 stuks van product 'A' (prijs €10) en 1 stuk van product 'B' (prijs €20) in de winkelmand.	De gebruiker navigeert naar de checkoutpagina.	De 'Order Summary' sectie toont de totale prijs als €40 (2 * €10 + 1 * €20).	e2e
AC-009-1	REQ-009	Een gebruiker opent de applicatie op een desktop browser.	De homepage wordt geladen.	De layout van de homepage is geoptimaliseerd voor een desktop schermresolutie, met elementen die goed gepositioneerd en leesbaar zijn.	e2e
AC-009-2	REQ-009	Een gebruiker opent de applicatie op een tablet browser.	De productlijstpagina wordt geladen.	De layout van de productlijstpagina past zich aan de tablet schermresolutie aan, met een aangepast aantal kolommen en leesbare elementen.	e2e
AC-009-3	REQ-009	Een gebruiker opent de applicatie op een mobiele browser.	De productdetailpagina wordt geladen.	De layout van de productdetailpagina is geoptimaliseerd voor een mobiel scherm, met elementen die verticaal gestapeld zijn en	e2e

AC-ID	REQ	Gegeven	Wanneer	Dan	Testtype
				gemakkelijk te navigeren met één hand.	
AC-009-4	REQ-009	Een gebruiker opent de applicatie op een mobiele browser.	De gebruiker probeert te scrollen door een lange lijst van producten.	Het scrollen is soepel en responsief, zonder haperingen of vertragingen.	e2e
AC-010-1	REQ-010	De applicatie is onder normale belasting (bv. 100 gelijktijdige gebruikers).	Een gebruiker navigeert naar de productlijstpagina (GET /api/products).	De productlijstpagina laadt volledig binnen 2 seconden.	integration
AC-010-2	REQ-010	De applicatie is onder verhoogde belasting (bv. 500 gelijktijdige gebruikers).	Een gebruiker navigeert naar de productlijstpagina (GET /api/products).	De productlijstpagina laadt binnen een acceptabele tijd (bv. maximaal 5 seconden), hoewel de 2-seconden eis mogelijk niet gehaald wordt.	integration
AC-011-1	REQ-011	Een nieuwe gebruiker registreert zich met het wachtwoord 'SecureP@ssw0rd1'.	De registratie wordt verwerkt en de gebruikersgegevens worden opgeslagen in de database.	Het wachtwoordveld in de database voor deze gebruiker bevat een gehasht representatie van 'SecureP@ssw0rd1' (bv. met bcrypt), en niet het platte tekst wachtwoord.	integration
AC-011-2	REQ-011	Een gebruiker probeert in te loggen met het correcte wachtwoord 'SecureP@ssw0rd1'.	Het ingevoerde wachtwoord wordt vergeleken met het opgeslagen gehashte wachtwoord.	De login is succesvol (HTTP 200) omdat het gehashte wachtwoord overeenkomt met het ingevoerde wachtwoord.	integration
AC-011-3	REQ-011	Een gebruiker probeert in te loggen met een incorrect wachtwoord 'WrongP@ssw0rd1'.	Het ingevoerde wachtwoord wordt vergeleken met het opgeslagen gehashte wachtwoord.	De login faalt (bv. HTTP 401) omdat het gehashte wachtwoord niet overeenkomt met het ingevoerde wachtwoord.	integration

13. Traceability Matrix

REQ	Backend	Frontend	Tests
REQ-001	ProductController, ProductService, ProductRepository, CartController, CartService, OrderController, OrderService	ProductListPage, ProductDetailPage, CartPage, CheckoutPage, OrderHistoryPage	Testen van het zoeken en filteren van producten.; Testen van het toevoegen van producten aan de winkelmand.; Testen van het afrekenproces.; Testen van het bekijken van de bestelgeschiedenis.
REQ-002	AdminProductController, AdminProductService, AdminOrderController, AdminOrderService, ProductRepository, OrderRepository	AdminProductListPage, AdminCreateProductPage, AdminEditProductPage, AdminOrderListPage, AdminOrderDetailPage	Testen van het aanmaken, bewerken en verwijderen van producten door admins.; Testen van het bekijken en beheren van de voorraad door admins.; Testen van het bekijken en beheren van bestellingen door admins.
REQ-003		ProductCard	Visuele inspectie van de productcard stijl en de zichtbaarheid en functionaliteit van CTA-knoppen.
REQ-004	OrderController, OrderService	CheckoutPage, OrderSummary	Testen of het besteloverzicht en de order summary correct worden weergegeven tijdens het checkout proces.

REQ	Backend	Frontend	Tests
REQ-005		HomePage, HeroSection	Visuele inspectie van de homepage hero sectie, inclusief titel, ondertitel, CTA-knoppen en afbeelding.
REQ-006	ProductController, ProductService, ProductRepository	ProductDetailPage, ProductImageGallery, ProductDetails, AddToCartButton, FavoriteButton	Testen of alle productinformatie (afbeelding, titel, beschrijving, opties, prijs) correct wordt weergegeven op de productdetailpagina.; Testen van de functionaliteit van de 'toevoegen aan winkelmand' en 'favoriet' knoppen.
REQ-007	OrderController, OrderService	CheckoutPage, ShippingForm, PaymentMethodSelector	Testen van de invoervelden voor verzendgegevens en de selectie van betaalmethoden (Bancontact, Creditcard) in de checkout.
REQ-008	OrderController, OrderService	CheckoutPage, OrderSummary	Testen of de order summary correct de bestelde items (naam, aantal, prijs) en de totale prijs weergeeft.
REQ-009		HomePage, ProductListPage, ProductDetailPage, CartPage, CheckoutPage, OrderHistoryPage, LoginPage, RegisterPage, AdminDashboardPage	Testen van de layout en functionaliteit van de applicatie op desktop, tablet en mobiele apparaten.
REQ-010	ProductController, ProductService	ProductListPage, ProductCardList	Metten van de laadtijd van het productoverzicht onder normale belasting om te verifiëren dat deze binnen 2 seconden laadt.
REQ-011	AuthService, PasswordHasher		Testen van het registratieproces en controleren of wachtwoorden gehasht worden opgeslagen in de database (via inspectie van de database of een gecontroleerde API-call).